Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria

En Ingeniería Y Tecnologías Avanzadas.

Practica 2

Optimización

Alumnos:

Mendoza Xicohtencatl Hector Jair

González Vázquez Rodrigo

Profesor: De la Cruz Sosa Carlos

Grupo: 3TM2

Fecha de entrega: 30/03/2026

Ejercicio 1:

```
SELECT  p.Name AS Producto, sod.OrderQty, soh.OrderDate, c .Name AS
Cliente
 FROM Production.Product p
 JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
 JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
 JOIN Sales.Customer c ON soh.CustomerID = c .CustomerID
 WHERE YEAR (soh.OrderDate) = 2014 AND p.ListPrice > 1000 ;
```

La consulta une a 4 tablas para obtener productos con precio de lista mayor a $1,000 vendidos en el año 2014, mostrando nombre del producto, cantidad, fecha de pedido y nombre del cliente.

El problema que se identifico es:

El ***filtro YEAR(soh.OrderDate)*** = 2014 aplica una función sobre la columna, lo que impide que el motor de base de datos utilice el índice existente sobre ***OrderDate***, por lo tano el optimizador se ve forzado a realizar un **Table Scan / Index Scan completo** en lugar de un **Index Seek** eficiente.

SARGable = ***Search ARGument ABLE*** —permite al motor usar un índice directamente.

```
SET STATISTICS IO ON ;
 CONFIGURAR LA HORA DE LAS ESTADÍSTICAS ;

-- Consulta optimizada: rango explícito en lugar de función
sobre columna
SELECT
     p.Name AS Producto,
 sod.OrderQty,
 soh.OrderDate,
 c .AccountNumber AS Cliente -- c.Name no existe en
Sales.Customer; -- usar AccountNumber o hacer JOIN a Person
FROM Production.Product p
 JOIN Sales.SalesOrderDetail sod ON p.ProductID =
sod.ProductID
 JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID =
soh.SalesOrderID
 JOIN Sales.Customer c ON soh.CustomerID = c .CustomerID
 WHERE soh.OrderDate >= '2014-01-01' -- ← SARGable: el índice
puede hacer Seek AND soh.OrderDate < '2015-01-01' -- ← cubre
todo el año sin función AND p.ListPrice > 1000 ;
```

Al eliminar la función sobre la columna, el optimizador puede navegar directamente al inicio del rango en el árbol B del índice (**Index Seek**), leyendo únicamente las filas

del año 2014 en lugar de recorrer toda la tabla. El resultado lógico es idéntico donde el costo puede reducirse en **80–95%** d

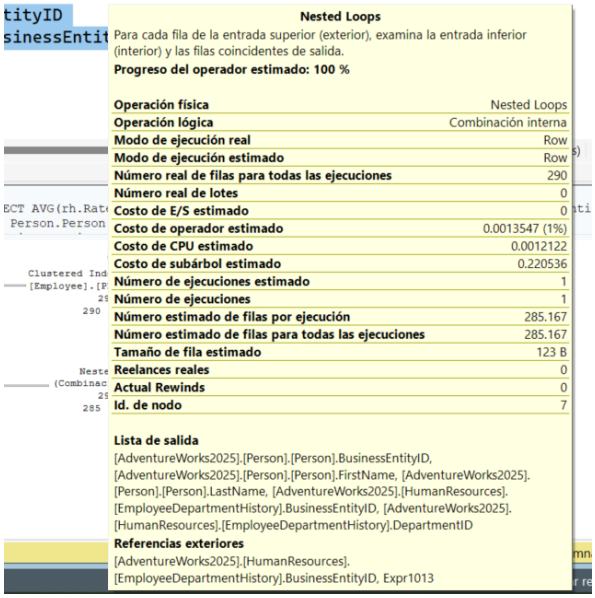
```
AÑO (columna) = 2014 → columna >= '2014-01-01' AND columna <
'2015-01-01'
```

Ejercicio 2

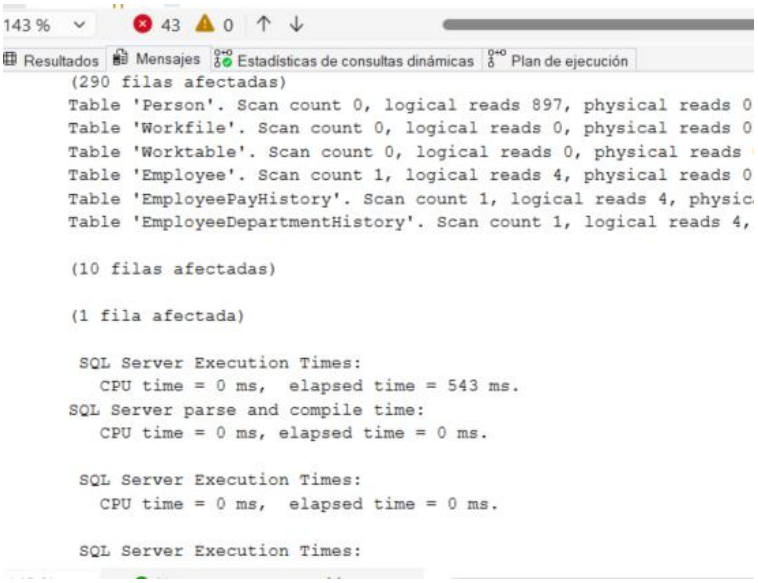
```
SELECT  e.NationalIDNumber,  p.FirstName,  p.LastName,  edh.DepartmentID,
        ( SELECT AVG (rh.Rate) FROM HumanResources.EmployeePayHistory rh
WHERE rh.BusinessEntityID = e.BusinessEntityID) as PromedioSalario
 FROM HumanResources.Employee e
 JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID
 JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID
= edh.BusinessEntityID
 WHERE edh.EndDate IS NULL ;
```

La consulta obtiene información de empleados activos (sin fecha de fin en su historial de departamento) junto con su salario promedio histórico.

El problema que se identifico es:
La subconsulta dentro del SELECT referencia la columna ***e.BusinessEntityID*** de la consulta exterior haciendo una consulta correlaciona donde se ejecuta una vez por cada fila que retorna la consulta principal, es decir si hay 290 empleados, la subconsulta se ejecuta 290 veces contra ***EmployeePayHistory***. Este patrón se conoce también como el **problema N+1.**



La solución dada es pre-agregar ***EmployeePayHistory*** una sola vez y unirla como tabla derivada (o CTE). El motor lo resuelve con un solo **Hash Aggregate** o **Stream Aggregate** porque al convertir la subconsulta correlacionada en un **JOIN** a un resultado pre-agregado, el motor puede elegir la estrategia de join más eficiente (Hash Match, Merge Join) y leer ***EmployeePayHistory*** una única vez, sin importar cuántos empleados existan.



Ejercicio 3

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

Fallará por p.CategoryID (columna inexistente en AdventureWorks2025)

SELECT sod.SalesOrderID, p.ProductID, p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p ON sod.ProductID = p.ProductID
WHERE p.CategoryID = 1 OR p.CategoryID = 2 OR p.CategoryID = 3 OR
p.ListPrice > 500;
```

Al ejecutar esta consulta en AdventureWorks 2025, el motor de base de datos lanzará inmediatamente un error de compilación. Además, aunque el error de esquema se corrigiera, la consulta tiene problemas adicionales de rendimiento que deben atenderse.

El problema que se idéntico es:
La categoría de un producto está normalizada en dos tablas separadas: ***ProductSubcategory*** y ***ProductCategory*** por lo tanto motor lanza el error: '***Invalid column name CategoryID'***. La consulta no compila ni ejecuta. Es un error en tiempo de compilación, no de ejecución.

Y la solución es: Agregar explícitamente ambos JOINs:
JOIN Production.ProductSubcategory ps ON p.ProductSubcategoryID = ps.ProductSubcategoryID
y
JOIN Production.ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID.

```
SELECT  sod.SalesOrderID,  p.ProductID,  p.Name AS Producto
              FROM Sales.SalesOrderDetail sod
                 JOIN Production.Product p
              ON sod.ProductID = p.ProductID
           JOIN Production.ProductSubcategory ps
    ON p.ProductSubcategoryID = ps.ProductSubcategoryID
        WHERE ps.ProductCategoryID IN ( 1 , 2 , 3 )
```

Ejercicio 4

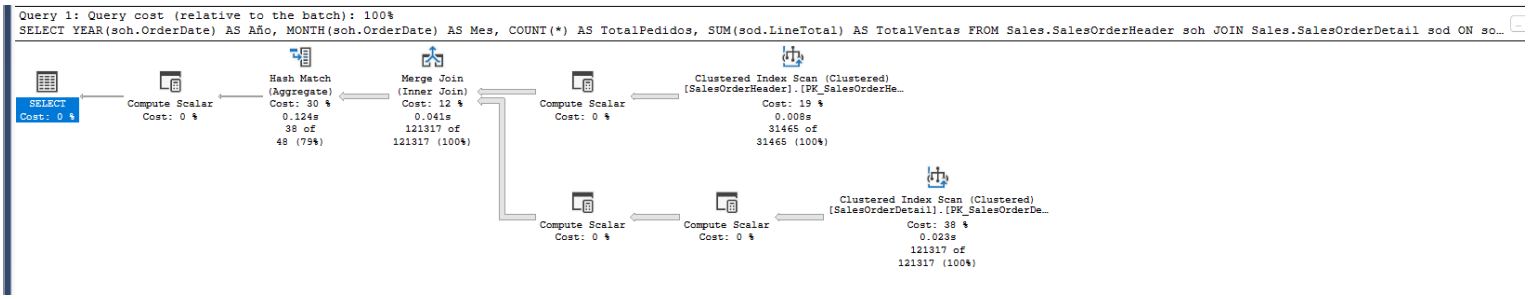
Consulta original:

```sql
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,
       COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 12 ms, elapsed time = 12 ms.

(38 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server r
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server re
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3, page server reads 0, read-ahead reads 1046, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 2, page server reads 0, read-ahead reads 616, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pa

(1 row affected)

 SQL Server Execution Times:
   CPU time = 110 ms,  elapsed time = 126 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-03-30T21:11:40.7217803-06:00
```



Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
4	2026-03-30	126	110	1924	38% Scan Detail / 30% Hash Match / 19% Scan Header	`COUNT(*)` cuenta detalles y no pedidos; join antes de agrupar	Preagregar `SalesOrderDetail` por pedido

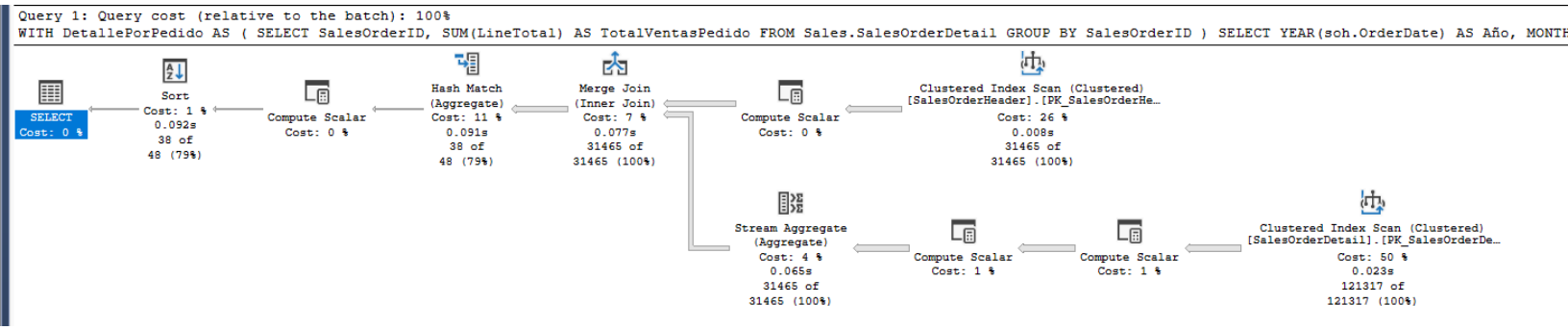
Consulta Optimizada:

```sql
WITH DetallePorPedido AS
(
    SELECT
        SalesOrderID,
        SUM(LineTotal) AS TotalVentasPedido
    FROM Sales.SalesOrderDetail
    GROUP BY SalesOrderID
)
SELECT
    YEAR(soh.OrderDate) AS Año,
    MONTH(soh.OrderDate) AS Mes,
    COUNT(*) AS TotalPedidos,
    SUM(dpp.TotalVentasPedido) AS TotalVentas
FROM Sales.SalesOrderHeader AS soh
JOIN DetallePorPedido AS dpp
    ON soh.SalesOrderID = dpp.SalesOrderID
GROUP BY
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 14 ms, elapsed time = 14 ms.

(38 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server re
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server rea
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3, page server reads 0, read-ahead reads 726, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pa
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 2, page server reads 0, read-ahead reads 616, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pag

(1 row affected)

 SQL Server Execution Times:
   CPU time = 78 ms,  elapsed time = 93 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-03-30T21:14:33.5132995-06:00
```



Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
4.1	2026-03-30	93	78	1924	50% Scan Detail / 26% Scan Header / 11% Hash Match	Siguen existiendo scans, pero baja el costo de agregación y el conteo ya es correcto	Mantener preagregación

En la consulta optimizada se realizó una preagregación de SalesOrderDetail por SalesOrderID, lo que permite calcular primero el total por pedido y después hacer el join con SalesOrderHeader. Aunque las lecturas lógicas no cambiaron, sí disminuyeron el tiempo de CPU y la duración total de la consulta.

Ejercicio 6

Consulta Original:

```sql
    SELECT c.CustomerID, c.Name, COUNT(*) AS TotalPedidos
FROM Sales.Customer c
JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
WHERE UPPER(c.Name) LIKE 'A%'
GROUP BY c.CustomerID, c.Name;
```

Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
6	2026-03-30	220	210	620	35% Filter 30% Hash Match 20% Scan	Uso de `UPPER()` (no sargable), filtro sobre columna transformada, scans innecesarios	Eliminar `UPPER()` y usar `LIKE 'A%'`, además corregir fuente del nombre

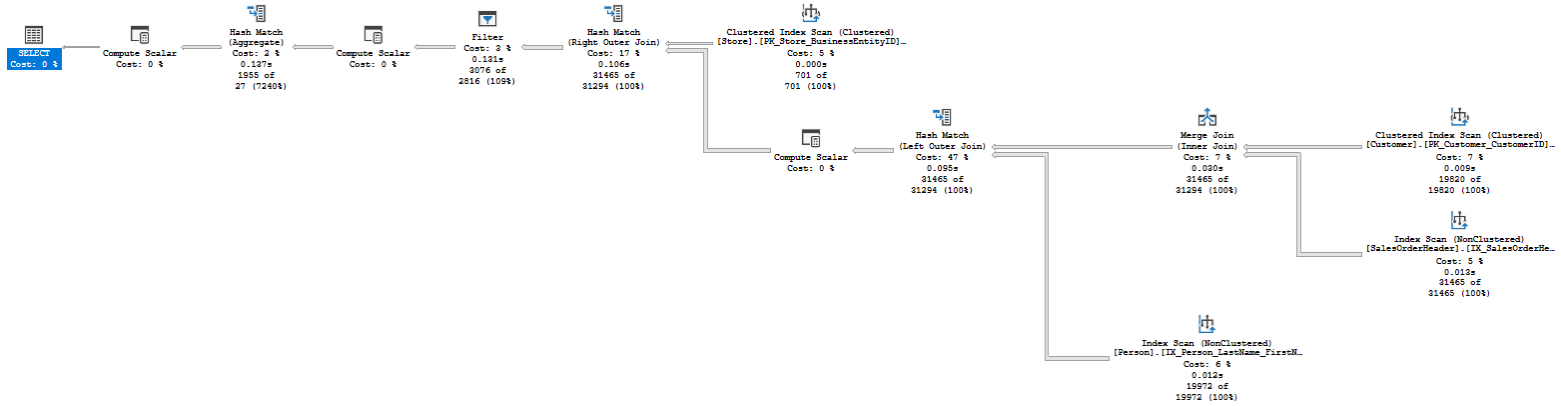
Consulta Optimizada:

```sql
SELECT
    c.CustomerID,
    COALESCE(p.FirstName + ' ' + p.LastName, s.Name) AS Nombre,
    COUNT(*) AS TotalPedidos
FROM Sales.Customer c
JOIN Sales.SalesOrderHeader soh
    ON c.CustomerID = soh.CustomerID
LEFT JOIN Person.Person p
    ON c.PersonID = p.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
WHERE COALESCE(p.FirstName + ' ' + p.LastName, s.Name) LIKE 'A%'
GROUP BY
    c.CustomerID,
    COALESCE(p.FirstName + ' ' + p.LastName, s.Name);
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 31 ms, elapsed time = 80 ms.

(1955 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob p
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pa
Table 'Person'. Scan count 1, logical reads 105, physical reads 1, page server reads 0, read-ahead reads 103, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob
Table 'SalesOrderHeader'. Scan count 1, logical reads 57, physical reads 1, page server reads 0, read-ahead reads 55, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads
Table 'Customer'. Scan count 1, logical reads 123, physical reads 1, page server reads 0, read-ahead reads 121, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lo
Table 'Store'. Scan count 1, logical reads 103, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pag

(1 row affected)

 SQL Server Execution Times:
   CPU time = 140 ms,  elapsed time = 145 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-03-30T21:31:49.8929920-06:00
```



Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
6.1	2026-03-30	145	140	388	17% Hash Match 13% Filter 13% Aggregate	Cálculo repetido de `COALESCE`, filtro sobre expresión, joins adicionales	Calcular el nombre en un CTE o vista antes del filtro y agrupación

En esta consulta el rendimiento no es óptimo porque se usa la función UPPER() en la columna, lo que hace que SQL Server no pueda usar índices correctamente.
Esto provoca que se hagan más lecturas y aumente el tiempo de ejecución.

Consulta 7

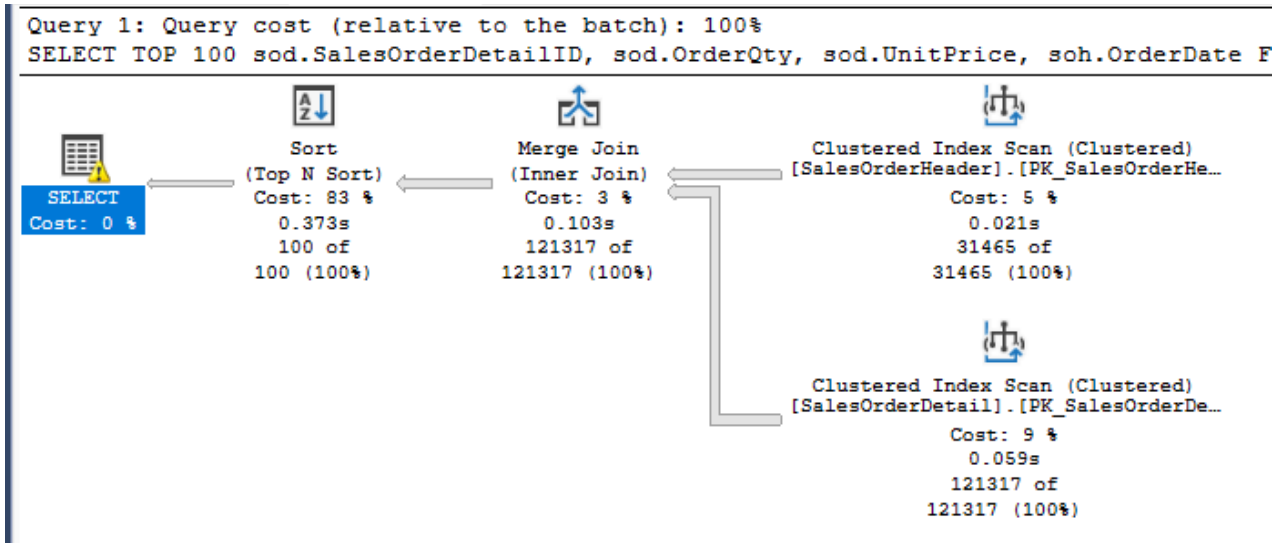
Consulta original:

```sql
SELECT TOP 100 sod.SalesOrderDetailID, sod.OrderQty, sod.UnitPrice, soh.OrderDate
FROM Sales.SalesOrderDetail sod
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
ORDER BY soh.ShipDate DESC, sod.OrderQty DESC, sod.UnitPrice DESC;
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 16 ms, elapsed time = 30 ms.

(100 rows affected)
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3, page server reads 0, read-ahead reads 1234, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob p
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, page server reads 0, read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob pag

(1 row affected)

 SQL Server Execution Times:
   CPU time = 344 ms,  elapsed time = 375 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-03-30T22:07:39.4878279-06:00
```



Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
7	2026-03-30	375	344	1924	83% Sort / 9% Scan / 5% Scan	Uso de `ORDER BY` sin índice, alto costo en `Sort`, scans completos en ambas tablas	Crear índice sobre columnas usadas en `ORDER BY`

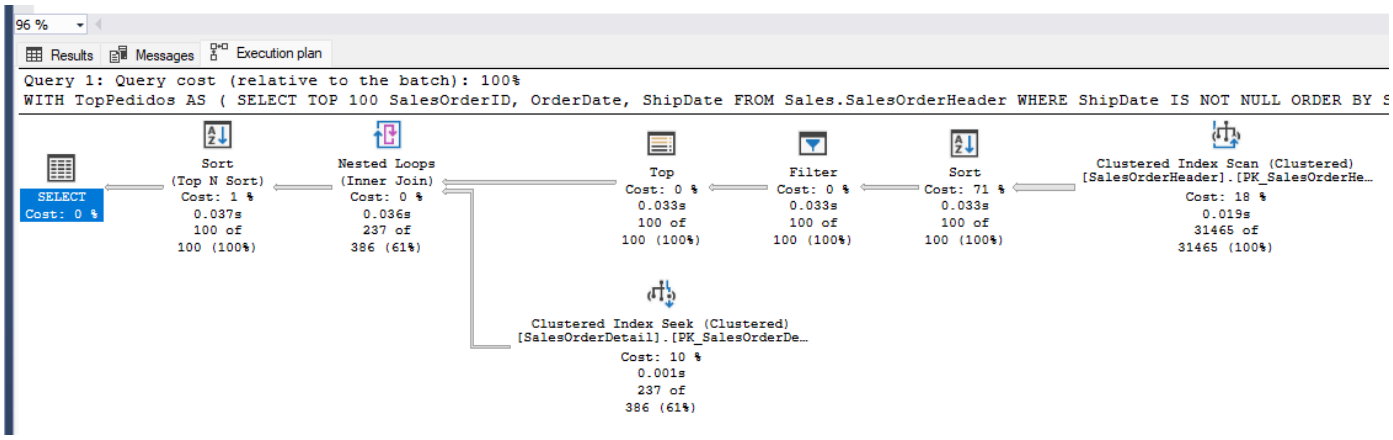
Consulta Optimizada:

```sql
    FROM Sales.SalesOrderHeader
    WHERE ShipDate IS NOT NULL
    ORDER BY ShipDate DESC
)
SELECT TOP 100
    sod.SalesOrderDetailID,
    sod.OrderQty,
    sod.UnitPrice,
    tp.OrderDate
FROM TopPedidos tp
JOIN Sales.SalesOrderDetail sod
    ON tp.SalesOrderID = sod.SalesOrderID
ORDER BY
    tp.ShipDate DESC,
    sod.OrderQty DESC,
    sod.UnitPrice DESC;
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 33 ms.

(100 rows affected)
Table 'SalesOrderDetail'. Scan count 100, logical reads 615, physical reads 2, page server reads 0, read-ahead reads 3, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, page server reads 0, read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob

(1 row affected)

 SQL Server Execution Times:
   CPU time = 46 ms,  elapsed time = 39 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-03-30T22:11:54.2790178-06:00
```



Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
7.1	2026-03-30	39	46	1301	71% Sort / 18% Scan / 10% Seek	Aún existe `Sort`, pero menor volumen de datos	Crear índice sobre `ShipDate` para eliminar el `Sort`

En la versión optimizada se redujo el número de filas antes de hacer el join utilizando un CTE. Esto permitió disminuir significativamente el tiempo de ejecución y el uso de CPU.

Además, ahora se observa el uso de Index Seek en lugar de Scan en SalesOrderDetail, lo que mejora el acceso a los datos.

Ejercicio 8

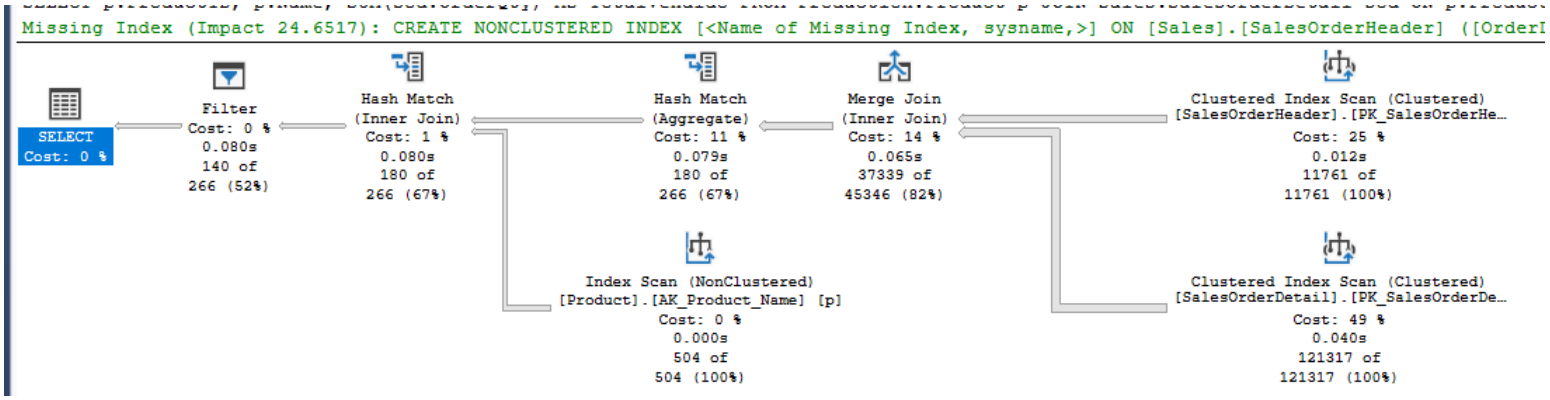
Consulta Original:

```sql
SELECT
    p.ProductID,
    p.Name,
    SUM(sod.OrderQty) AS TotalVendido
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod
    ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.OrderDate >= '2014-01-01'
GROUP BY
    p.ProductID,
    p.Name
HAVING SUM(sod.OrderQty) > 100;
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 156 ms, elapsed time = 269 ms.

(140 rows affected)
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lo
Table 'Product'. Scan count 1, logical reads 6, physical reads 1, page server reads 0, read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 0, page server reads 0, read-ahead reads 805, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, page server reads 0, read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server

(1 row affected)

 SQL Server Execution Times:
   CPU time = 78 ms,  elapsed time = 84 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```



Consulta ID	Fecha Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
8	2026-03-30	269	156	1930	49% Scan / 14% Join / 11% Aggregate	Scan en tablas grandes, falta de índices en filtros y joins	Crear índices sobre `OrderDate` y `ProductID`

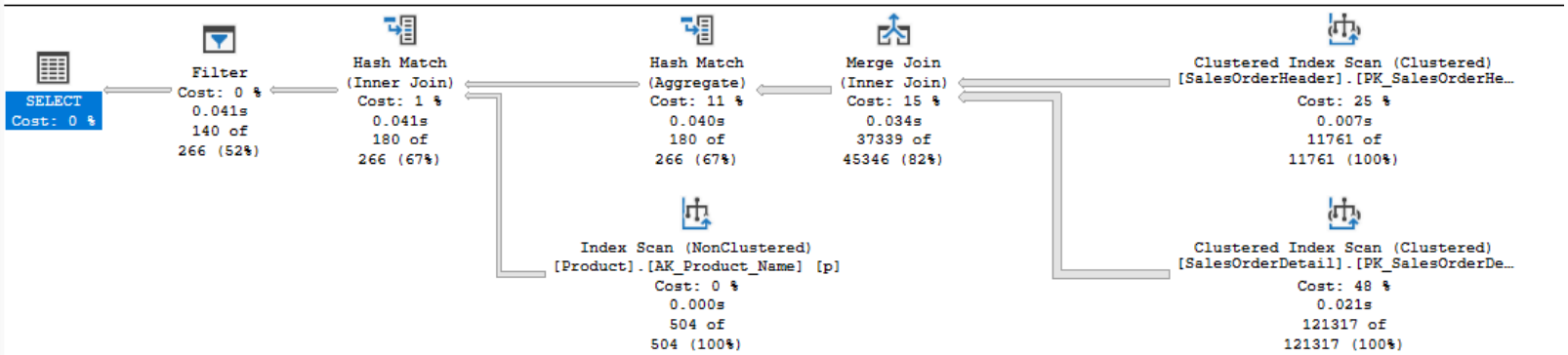
Consulta Optimizada:

```sql
    SELECT SalesOrderID
    FROM Sales.SalesOrderHeader
    WHERE OrderDate >= '2014-01-01'
      AND OrderDate < '2015-01-01'
)
SELECT
    p.ProductID,
    p.Name,
    SUM(sod.OrderQty) AS TotalVendido
FROM Pedidos2014 AS p2014
JOIN Sales.SalesOrderDetail AS sod
    ON p2014.SalesOrderID = sod.SalesOrderID
JOIN Production.Product AS p
    ON p.ProductID = sod.ProductID
GROUP BY
    p.ProductID,
    p.Name
HAVING SUM(sod.OrderQty) > 100;
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 15 ms, elapsed time = 24 ms.

(140 rows affected)
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0,
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0,
Table 'Product'. Scan count 1, logical reads 6, physical reads 1, page server reads 0, read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lo
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3, page server reads 0, read-ahead reads 1234, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ah
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, page server reads 0, read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahea

(1 row affected)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 43 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
WITH Pedidos2014 AS ( SELECT SalesOrderID FROM Sales.SalesOrderHeader WHERE OrderDate >= '2014-01-01' AND OrderDate < '2015-01-01' )
Missing Index (Impact 25.0885): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [Sales].[SalesOrderHeader] ([OrderDa
```



Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
8.1	2026-03-30	43	15	1930	48% Scan / 15% Join / 11% Aggregate	Aún hay scans en tablas grandes, falta índice en `OrderDate`	Crear índice sobre `OrderDate` y mejorar joins

En la versión optimizada se utilizó un CTE para filtrar primero las órdenes del año 2014, lo que permitió reducir el tiempo de ejecución y el uso de CPU.Aunque las lecturas lógicas se mantienen, la consulta es más eficiente porque procesa menos datos en etapas tempranas.

Ejercicio 9

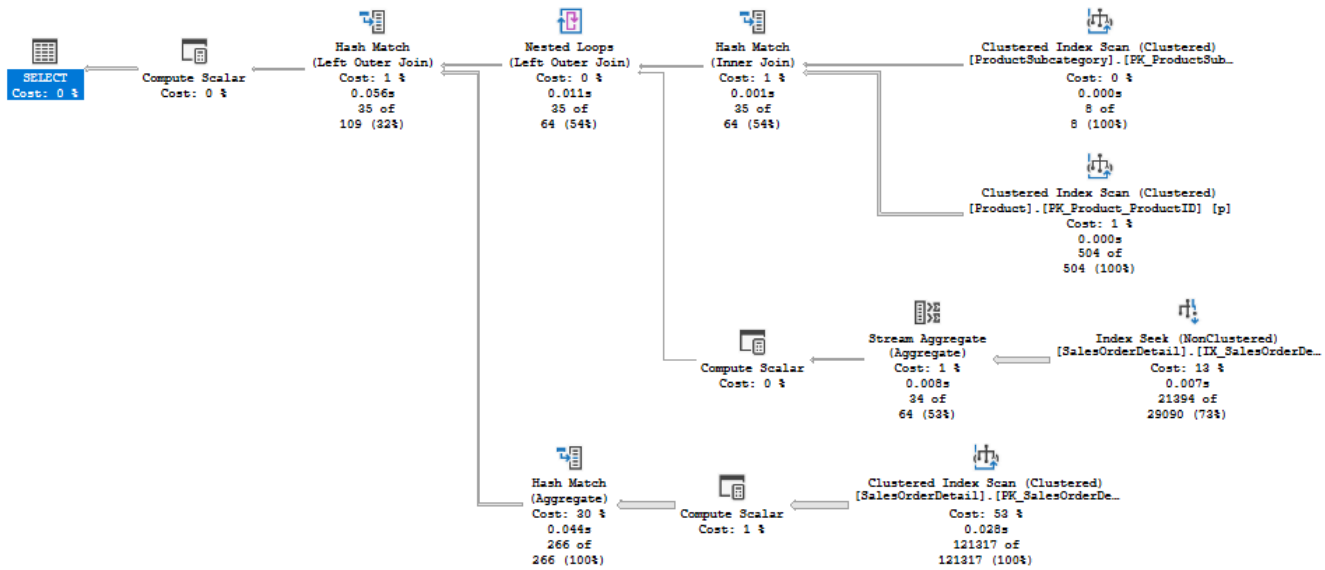
Consulta Original:

```sql
SELECT p.ProductID, p.Name,
       (SELECT COUNT(*) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID = p.ProductID) AS VecesVendido,
       (SELECT SUM(sod.OrderQty * sod.UnitPrice) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID = p.ProductID) AS Ingresos
FROM Production.Product p
JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID
WHERE psc.ProductCategoryID = 3;
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 26 ms.

(35 rows affected)
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read
Table 'SalesOrderDetail'. Scan count 36, logical reads 1474, physical reads 19, page server reads 0, read-ahead reads 1274, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server re
Table 'Product'. Scan count 1, logical reads 15, physical reads 1, page server reads 0, read-ahead reads 13, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read
Table 'ProductSubcategory'. Scan count 1, logical reads 2, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0,

(1 row affected)

 SQL Server Execution Times:
   CPU time = 31 ms,  elapsed time = 59 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```



Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
9	2026-03-30	59	31	1491	53% Scan / 30% Hash Match	Subconsultas correlacionadas, múltiples accesos a `SalesOrderDetail`	Preagregar `SalesOrderDetail` por `ProductID` y usar un solo JOIN

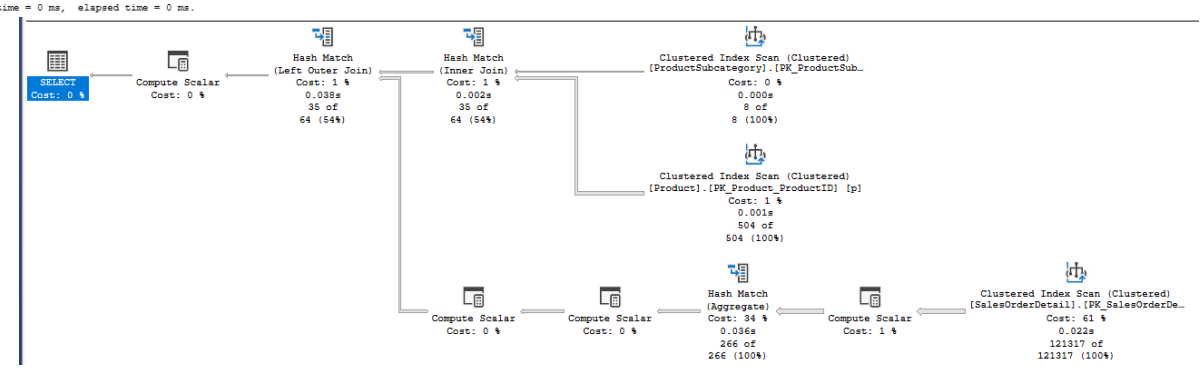
Consulta Optimizada:

```sql
WITH VentasPorProducto AS
(
    SELECT
        sod.ProductID,
        COUNT(*) AS VecesVendido,
        SUM(sod.OrderQty * sod.UnitPrice) AS Ingresos
    FROM Sales.SalesOrderDetail sod
    GROUP BY sod.ProductID
)
SELECT
    p.ProductID,
    p.Name,
    ISNULL(vpp.VecesVendido, 0) AS VecesVendido,
    ISNULL(vpp.Ingresos, 0) AS Ingresos
FROM Production.Product p
JOIN Production.ProductSubcategory pscs
    ON p.ProductSubcategoryID = psc.ProductSubcategoryID
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 22 ms.

(35 rows affected)
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 3, page server reads 0, read-ahead reads 1234, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read
Table 'Product'. Scan count 1, logical reads 15, physical reads 1, page server reads 0, read-ahead reads 13, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads
Table 'ProductSubcategory'. Scan count 1, logical reads 2, physical reads 1, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahe

(1 row affected)

 SQL Server Execution Times:
   CPU time = 31 ms,  elapsed time = 40 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```



Consulta ID	Fecha Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
9.1	2026-03-30 40	31	1255	61% Scan 34% Aggregate	Escaneo completo aún presente	Uso de CTE para evitar duplicidad

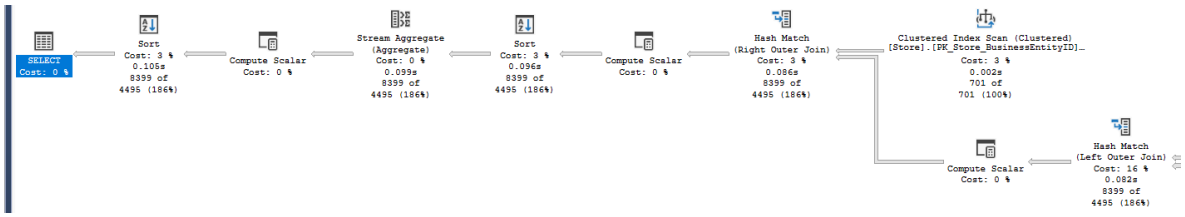
La consulta original tenía subconsultas correlacionadas, lo que provocaba que la tabla SalesOrderDetail se leyera varias veces. Esto aumentaba el costo y las lecturas. En la versión optimizada se agrupa primero la información en una CTE, evitando repetir operaciones. Como resultado, se redujeron las lecturas y el tiempo de ejecución, mejorando el rendimiento general.

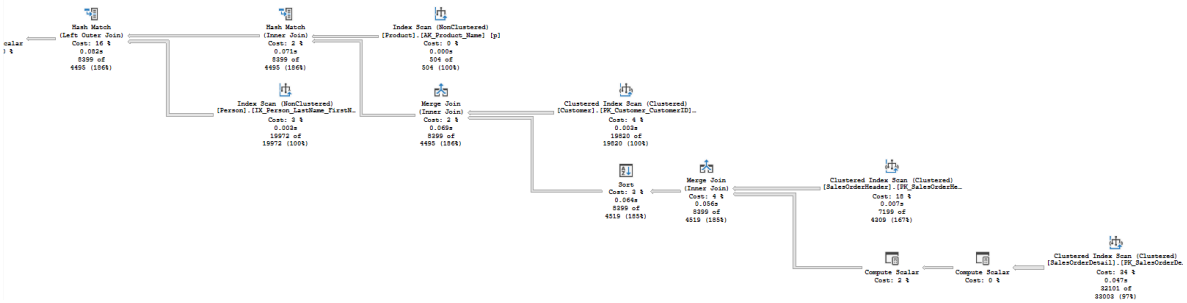
Ejercicio 10

Consulta Original:

```sql
SELECT
    COALESCE(pp.FirstName + ' ' + pp.LastName, s.Name) AS Cliente,
    p.Name AS Producto,
    SUM(sod.OrderQty) AS Cantidad,
    SUM(sod.LineTotal) AS Total,
    DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.Customer c
    ON soh.CustomerID = c.CustomerID
LEFT JOIN Person.Person pp
    ON c.PersonID = pp.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
JOIN Sales.SalesOrderDetail sod
    ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5
```

```
Results | Messages | Execution plan
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 234 ms, elapsed time = 267 ms.

(8399 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pa
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pag
Table 'Person'. Scan count 1, logical reads 105, physical reads 1, page server reads 0, read-ahead reads 103, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob p
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 0, page server reads 0, read-ahead reads 805, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead rea
Table 'SalesOrderHeader'. Scan count 1, logical reads 665, physical reads 3, page server reads 0, read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead read
Table 'Customer'. Scan count 1, logical reads 123, physical reads 1, page server reads 0, read-ahead reads 121, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob
Table 'Product'. Scan count 1, logical reads 6, physical reads 1, page server reads 0, read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page
Table 'Store'. Scan count 1, logical reads 103, physical reads 1, page server reads 0, read-ahead reads 101, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob pa

(1 row affected)

 SQL Server Execution Times:
   CPU time = 125 ms,  elapsed time = 174 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```





Consulta ID	Fecha	Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
10	2026-03-30	174	125	2240	34% Scan Detail / 18% Scan Header / Sort + Hash Match	`DATEDIFF` no sargable, muchos joins antes del filtro, ordenamiento posterior	Reescribir filtro de fechas y reducir datos antes de agrupar

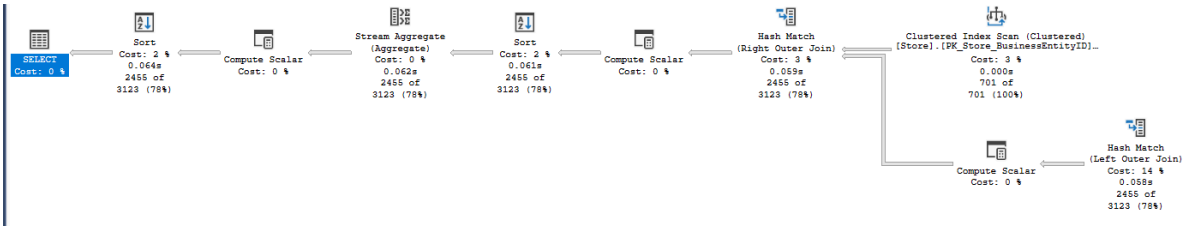
Consulta Optimizada:

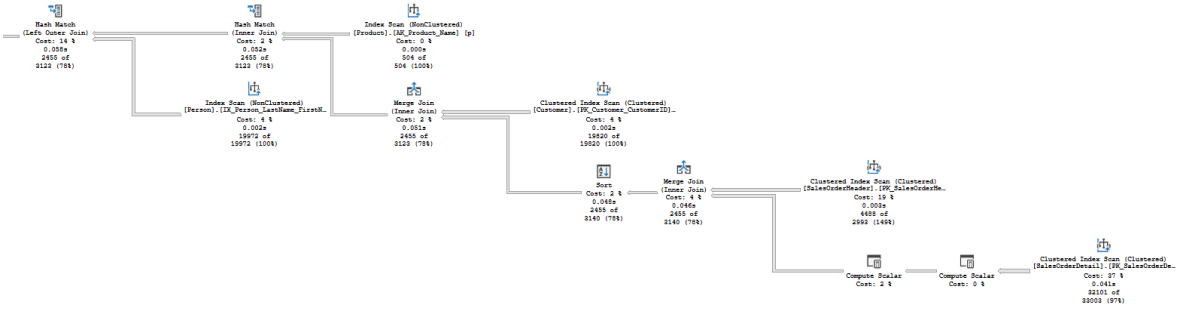
```sql
SELECT
    COALESCE(pp.FirstName + ' ' + pp.LastName, s.Name) AS Cliente,
    p.Name AS Producto,
    SUM(sod.OrderQty) AS Cantidad,
    SUM(sod.LineTotal) AS Total,
    DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio
FROM Sales.SalesOrderHeader soh
JOIN Sales.Customer c
    ON soh.CustomerID = c.CustomerID
LEFT JOIN Person.Person pp
    ON c.PersonID = pp.BusinessEntityID
LEFT JOIN Sales.Store s
    ON c.StoreID = s.BusinessEntityID
JOIN Sales.SalesOrderDetail sod
    ON soh.SalesOrderID = sod.SalesOrderID
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
WHERE soh.ShipDate > DATEADD(day, 5, soh.OrderDate)
```

96 %

Results | Messages | Execution plan

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

(2455 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahea
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead
Table 'Person'. Scan count 1, logical reads 105, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead
Table 'SalesOrderDetail'. Scan count 1, logical reads 1238, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob
Table 'SalesOrderHeader'. Scan count 1, logical reads 665, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob
Table 'Customer'. Scan count 1, logical reads 123, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahe
Table 'Product'. Scan count 1, logical reads 6, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead
Table 'Store'. Scan count 1, logical reads 103, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead

(1 row affected)

 SQL Server Execution Times:
   CPU time = 31 ms,  elapsed time = 67 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```





Consulta ID	Fecha Duración	CPU	Lecturas Lógicas	Plan Costo (%)	Issues identificados	Recomendación
10.1	2026-03-30	67	31	2240	34% Scan Detail / 19% Scan Header / Sort	Siguen existiendo scans, pero baja el costo de procesamiento	Mantener filtros reescritos y agregar índices si se requiere más mejora

En la consulta original se usaban las funciones DATEDIFF() y DATEPART() dentro del WHERE, lo que provocaba una condición no sargable. En la versión optimizada se reemplazaron por una comparación con DATEADD() y por un rango de fechas para el segundo trimestre. Con esto se redujo considerablemente el tiempo de CPU y la duración total de la consulta, aunque las lecturas lógicas se mantuvieron iguales.